cvopt — CV optimizer

cvopt builds a CV tailored to one job description from facts you have confirmed about yourself. You feed it your documents (CV, reference letters, diplomas); it extracts positions, records and claims, and you review every proposal before anything is stored. For each job description it then writes a JSON Resume CV that maximizes an ATS-style match score within two pages, an annotation file, a report, and PDFs.

Generated statements are labelled honestly. Every statement is *unmodified*, *rephrased*, *exaggerated* or *invented*, with a *provability* from `high` (backed by a reference letter, diploma or named reference) down to `none`. Hard facts (employers, titles, dates, education, languages, certificates, and every number and name in your claims) are never invented or altered. The annotated PDF marks exaggerated (`E`, amber) and invented (`I`, red) statements, so you decide what to keep before you send the clean version.

Setup

Requirements: Python 3.12 and uv. Everything else, including the typesetter and fonts, comes as Python packages.

```
git clone <this repo> && cd cv.auto
uv sync
export ANTHROPIC_API_KEY=sk-ant-...      # the key is read only from the environment
export CVOPT_USER=you@example.com        # or pass --user to every command
uv run cvopt version
```

Your data lives under `~/.local/share/cvopt` (one shared SQLite database, plus one directory per user named by a hash of the email, readable only by you). Text from your documents leaves your machine only in calls to the configured model provider.

Worked example: from your documents to an annotated PDF

1. Ingest your CV. cvopt reads PDF, DOCX, plain text and images (scans are transcribed by the model), extracts proposals and matches them against what you already have.

```
uv run cvopt ingest ~/Documents/cv.pdf --kind cv
```

It then walks you through each proposal:

```
[a]ccept  [r]eject  [e]dit  [s]kip  [A]ccept all remaining  [q]uit
```

Rejecting a position also rejects the claims tied to it. `q` keeps your decisions; resume with `cvopt review <batch>` (`cvopt review list` shows open batches). When nothing is pending, confirm *Apply now?* to store the accepted items. `--accept-all` skips the prompts.

2. Add evidence. A reference letter raises the claims it confirms to provability `high`. German Arbeitszeugnisse are read by their coded meaning: a passage graded only "satisfactory" ("zu unserer vollen Zufriedenheit") becomes a note, not evidence.

```
uv run cvopt ingest ~/Documents/reference-acme.pdf --kind reference_letter
uv run cvopt claim list                     # what you have, with provability
uv run cvopt claim add --type skill --text "Apache Airflow" --keywords "Airflow" --learnable
uv run cvopt evidence add <claim-id> --kind reference_person --reference "Dr. Jane Doe"
```

Ids show as their last 8 characters; any unique suffix of at least 6 characters works. `cvopt position|education|language|certificate list|add|edit|rm` and `cvopt profile show|set` manage the rest by hand.

3. Take in a job description. From a URL, a PDF or a text file:

```
uv run cvopt run new --url https://jobs.example.com/senior-data-scientist
# or: --pdf jd.pdf   or: --text jd.txt
```

This creates a run folder such as `~/.local/share/cvopt/users/<key>/runs/2026-10-24T1622-acme-senior-data-scientist/` with `jd.txt`, the original page, `requirements.json` and `run.json`. You may edit `requirements.json` (an importance, a keyword) before generating. Pages that cannot be read (errors, login walls) end with a message asking for `--pdf` or `--text`.

4. Generate.

```
uv run cvopt run generate 2026-10-24T1622     # a unique prefix of the folder name is enough
```

The first run makes many model calls (a few minutes); every answer is cached per user, so regenerating or rescoring is fast and reproducible. The run folder then holds:

File	Content
`cv.json`	The CV, valid JSON Resume, no annotations inside.
`annotations.json`	Per statement: fidelity, provability, source claims, requirements won, impact.
`report.json`, `report.csv`	Score, gaps and dropped claims, one row each.
`cv-annotated.pdf`	The CV with `E`/`I` marks and row numbers, and the report as appendix.
`candidates.json`	Every candidate statement and why it was or was not chosen.

`cvopt run go --url …` does steps 3 and 4 in one go.

5. Render the clean PDF to send, after editing `cv.json` if you like:

```
uv run cvopt render <run-folder>/cv.json -o ~/cv-acme.pdf
uv run cvopt render <run-folder>/cv.json --annotated     # re-create the annotated PDF
```

The default PDF layout is adapted from Atsthetic CV (MIT), bundled locally for offline rendering. It uses a single column, shaded section headings, and Source Sans 3. Set `[render] template = "classic"` in your config to use the previous plain layout. The selection applies to generation and standalone rendering, including annotated PDFs. Clean exports contain no annotation marks or report appendix.

6. Score an edited CV against the same requirements (the run's date is reused):

```
uv run cvopt score <run-folder>/cv.json --requirements <run-folder>/requirements.json \
    --annotations <run-folder>/annotations.json --report
```

7. Your data. `cvopt export -o backup.zip` writes all your records, documents and runs into one archive; `cvopt delete-user --confirm you@example.com` deletes all of them.

How the score works

Each requirement of the job description has a weight (must = 3, nice = 1) and is covered by the CV element that matches it best: exact keyword 1.0, synonym 0.9, same meaning 0.6, related 0.3, discounted by where the element sits (recent highlight 1.0, older highlight 0.85, headline or summary 0.8, skills list 0.7). The score is the weighted coverage, 0–100; each statement's impact is the points it wins, and the report lists the missing points per requirement as gaps. It approximates keyword-and-meaning ATS matchers; it is not any vendor's score.

Configuration

Optional: `~/.config/cvopt/config.toml` (or `--config FILE`, or `CVOPT_CONFIG`). Unknown keys are rejected. The defaults:

```
[app]
data_root = "~/.local/share/cvopt"
default_user = ""                 # --user and CVOPT_USER take precedence

[llm]
strong_model = "claude-sonnet-5-5"
fast_model = "claude-haiku-5-5"
max_concurrency = 4

[optimizer]
page_budget = 2
exaggeration_min_gain_pp = 1.0    # an exaggeration must beat its plain version by this

[scoring]
weight_must = 3
weight_nice = 1
recency_years = 5

[render]
template = "atsthetic"           # or "classic" for the previous plain layout
font = "Source Sans 3"
font_size_pt = 10
margin_mm = 18
```

See `src/cvopt/config.py` for every setting. Exit codes: 0 success, 1 input error, 2 model failure, 3 the rules or the page budget cannot be met, 4 internal error (details in `<data_root>/cvopt.log`, which never contains your documents' text).

Development

```
uv run pytest                    # offline: model answers come from tests/recordings
uv run ruff check && uv run ruff format --check && uv run mypy && uv run lint-imports
```

- `uv run pytest -m live` runs the golden scenarios against the real model (needs `ANTHROPIC_API_KEY`). `CVOPT_RECORD=1` adds new answers to the recordings; `CVOPT_RECORD=fresh` re-records all of them. A prompt-version bump makes the affected offline tests fail with `MissingRecording` until re-recorded.
- `tests/golden/` holds synthetic test data (invented people and companies); `uv run python tests/golden/build.py` rebuilds its PDFs from the Typst sources.
- `uv run python scripts/calibrate_layout.py` re-measures the `[layout]` constants after a template change.
- Schema changes: edit `src/cvopt/adapters/db/schema.py`, then `uv run alembic revision --autogenerate -m "..."`; migrations run automatically at startup.
- The design documents are in `masterplan/`.

Fonts: Source Sans 3, © Adobe, SIL Open Font License (`src/cvopt/resources/fonts/LICENSE.md`).